

KẾ HOẠCH TRIỂN KHAI MINI PROJECT

High-Dimensional Data Analysis trong Tin sinh học

Phân loại bạch cầu cấp ALL/AML từ dữ liệu biểu hiện gen (Golub et al., 1999)

Chủ đề: Bioinformatics · Design of experiment · Representation of data · Variable selection · Classification and prediction · Evaluation

Nhóm thực hiện: 6 thành viên (TV1 – TV6)

0. Tổng quan dự án

Tài liệu này là kế hoạch triển khai mini project của nhóm cho chủ đề **High-Dimensional Data Analysis**. Nhóm chọn bộ dữ liệu biểu hiện gen bệnh bạch cầu cấp của Golub và cộng sự (Science, 1999), gồm 72 mẫu bệnh nhân và 7129 probe (tương ứng 6817 gen người trên chip Affymetrix Hu6800). Đây là ví dụ kinh điển của bài toán $p \gg n$: số biến lớn gấp khoảng 100 lần số mẫu.

Kế hoạch được tổ chức theo bốn giai đoạn mà bộ môn yêu cầu, đồng thời lồng ghép sáu từ khóa của đề tài vào từng giai đoạn:

Giai đoạn	Nội dung chính	Từ khóa của đề tài được xử lý
1. Phân tích nghiệp vụ	Bối cảnh y học ALL/AML, câu hỏi nghiệp vụ, phát biểu bài toán, phân tích thiết kế thí nghiệm	Bioinformatics, Design of experiment
2. Thu thập và quản trị dữ liệu	Nguồn dữ liệu, kiến trúc lưu trữ, mô hình dữ liệu, ETL, kiểm tra chất lượng, tiền xử lý	Representation of data
3. Phân tích, khám phá, insight	EDA, giảm chiều, class discovery, chọn biến, phân loại, đánh giá	Variable selection, Classification and prediction, Evaluation
4. Trực quan hóa và storytelling	Nguyên tắc thiết kế hình, danh mục biểu đồ, cốt truyện trình bày, dashboard	Representation of data (trình bày kết quả)

Bảng 0.1. Ánh xạ giữa bốn giai đoạn triển khai và sáu từ khóa của đề tài

0.1. Mục tiêu dự án

- Xây dựng mô hình dự đoán ALL/AML chỉ dựa trên biểu hiện gen, với một tập gen nhỏ (khoảng 10–50 gen) và được đánh giá không thiên lệch.
- So sánh có hệ thống các phương pháp chọn biến (filter, wrapper, embedded) và các bộ phân loại trong điều kiện $p \gg n$.
- Chứng minh bằng thực nghiệm tác động của **selection bias** khi đánh giá sai cách, từ đó đề xuất quy trình đánh giá đúng (nested cross-validation).
- Tái lập (reproduce) các kết quả chính của Golub 1999 và so sánh với các phương pháp hiện đại.
- Truyền tải kết quả thành một câu chuyện dữ liệu rõ ràng, tuân thủ các nguyên tắc trực quan hóa đã học.

0.2. Sản phẩm bàn giao

- Báo cáo cuối kỳ (khoảng 25–35 trang) và bộ slide thuyết trình (15–20 slide).
- Kho mã nguồn (GitHub) có thể chạy lại toàn bộ từ dữ liệu thô đến hình vẽ, kèm README và file môi trường.
- Kho dữ liệu đã tổ chức theo tầng (raw → curated), kèm data dictionary.
- (Tùy chọn) Dashboard tương tác bằng Streamlit hoặc Plotly Dash để khám phá gen và mô hình.

Ngôn ngữ lập trình: **Python** (pandas, pyarrow, numpy, scipy, statsmodels, scikit-learn, matplotlib, seaborn, umap-learn, minisom, pytest; Streamlit nếu làm dashboard). Dữ liệu: bộ Golub trên Kaggle (biểu hiện gen, nhãn) kết hợp 6 cột mô tả mẫu từ OpenIntro.

1. Phân tích nghiệp vụ và bài toán

1.1. Bối cảnh y học

Bạch cầu cấp gồm hai nhóm chính: **ALL** (acute lymphoblastic leukemia, xuất phát từ tế bào tiền thân dòng lympho) và **AML** (acute myeloid leukemia, xuất phát từ dòng tủy). Dưới kính hiển vi, hai loại này trông rất giống nhau, nhưng phác đồ điều trị khác hẳn. Theo Golub et al., phác đồ ALL thường gồm corticosteroid, vincristine, methotrexate và L-asparaginase, còn AML dựa trên daunorubicin và cytarabine. Dùng nhầm phác đồ làm giảm mạnh tỷ lệ chữa khỏi và gây độc tính không cần thiết.

Thời điểm bài báo được công bố, chẩn đoán cần kết hợp hình thái học, hóa mô, miễn dịch và di truyền tế bào, mỗi xét nghiệm do một phòng lab chuyên biệt thực hiện. Không có xét nghiệm đơn lẻ nào đủ để kết luận. Bài báo còn kể một ca bệnh được chẩn đoán AML, nhưng mô hình biểu hiện gen cho điểm rất thấp với cả hai lớp; kiểm tra sau đó xác định bệnh nhân thực ra mắc sarcoma cơ vân (rhabdomyosarcoma) và phải đổi phác đồ. Đây là chi tiết rất mạnh để mở đầu phần storytelling.

1.2. Các bên liên quan và câu hỏi nghiệp vụ

Bên liên quan	Nhu cầu	Câu hỏi nghiệp vụ
Bác sĩ huyết học	Chẩn đoán nhanh, chính xác, có mức độ tin cậy	Có thể phân biệt ALL/AML chỉ bằng biểu hiện gen không? Khi nào mô hình nên trả lời "không chắc chắn"?
Phòng xét nghiệm	Xét nghiệm rẻ, chuẩn hóa được	Cần đo tối thiểu bao nhiêu gen? Quy trình chuẩn bị mẫu ảnh hưởng thế nào đến kết quả?
Nhà nghiên cứu sinh học	Hiểu cơ chế bệnh	Những gen nào phân biệt hai loại bệnh? Chúng có ý nghĩa sinh học không?
Nhà khoa học dữ liệu	Mô hình đáng tin cậy	Làm sao đánh giá mô hình trung thực khi chỉ có 72 mẫu và 7129 biến?

Bảng 1.1. Stakeholder và câu hỏi nghiệp vụ

1.3. Phát biểu bài toán dữ liệu

Gọi $\mathbf{X}$ là ma trận biểu hiện kích thước $n \times p$ với $n = 72$ mẫu và $p = 7129$ probe, $\mathbf{y} \in \{\text{ALL}, \text{AML}\}$ là nhãn chẩn đoán lâm sàng. Dự án gồm ba bài toán con, bám sát cấu trúc của bài báo gốc:

Bài toán	Loại học máy	Đầu vào → Đầu ra	Liên hệ với Golub 1999
Class prediction	Học có giám sát (phân loại nhị phân)	Biểu hiện gen của mẫu mới → ALL hoặc AML kèm độ tin cậy	Weighted voting với 50 gen, ngưỡng prediction strength 0.3
Variable selection	Chọn đặc trưng	7129 probe → tập nhỏ gen thông tin, ổn định	Chỉ số signal-to-noise $P(g,c)$ và neighborhood analysis
Class discovery	Học không giám sát (phân cụm)	Biểu hiện gen, không dùng nhãn → các cụm	SOM 2 cụm (ALL/AML) và 4 cụm (AML, T-ALL, B-ALL)

Bảng 1.2. Ba bài toán con của dự án

1.4. Thách thức của dữ liệu nhiều chiều ($p \gg n$)

- **Curse of dimensionality**: số mẫu cần để đạt cùng độ chính xác tăng theo hàm mũ với số biến; khoảng cách Euclid giữa các điểm dần mất ý nghĩa.

- **Overfitting:** với $p > n$, luôn tồn tại siêu phẳng tách hoàn hảo tập huấn luyện, kể cả khi nhãn là ngẫu nhiên.
- **Suy biến đại số:** ma trận $X^T X$ không khả nghịch, hồi quy logistic thường không hội tụ nếu không có regularization.
- **Kiểm định bội (multiple testing):** kiểm định 7129 gen ở mức $\alpha = 0.05$ cho kỳ vọng khoảng 356 dương tính giả nếu không hiệu chỉnh.
- **Selection bias:** chọn gen trên toàn bộ dữ liệu rồi mới cross-validation sẽ cho kết quả lạc quan giả tạo (Ambroise & McLachlan, 2002).

1.5. Thiết kế thí nghiệm (Design of Experiment)

1.5.1. Phân tích thiết kế thu thập mẫu của Golub 1999

Nhóm không tự tiến hành thí nghiệm sinh học, nhưng phải phân tích kỹ thiết kế của dữ liệu gốc vì nó quyết định cách diễn giải kết quả. Thông tin dưới đây lấy từ bài báo và phần chú thích (notes 13, 15, 23):

Đặc điểm	Tập huấn luyện (38 mẫu)	Tập kiểm tra độc lập (34 mẫu)
Phân bố lớp	27 ALL, 11 AML (mất cân bằng khoảng 71/29)	20 ALL, 14 AML
Loại mô	100% tủy xương (bone marrow)	24 tủy xương, 10 máu ngoại vi (peripheral blood)
Độ tuổi	ALL: trẻ em (DFCI); AML: người lớn (CALGB)	AML gồm cả người lớn và trẻ em
Nguồn/phòng lab	DFCI (ALL), CALGB (AML)	DFCI, St. Jude (SJCRH), CALGB, Children's Cancer Group
Quy trình chuẩn bị mẫu	Ficoll sedimentation; RNA tách bằng Trizol hoặc RNAqueous	Mẫu SJCRH dùng quy trình rất khác (hypotonic lysis, Qiagen)
Kiểm soát chất lượng	Loại mẫu có RNA < 15 μ g, lai yếu hoặc chip lỗi	Như tập huấn luyện

Bảng 1.3. Thiết kế mẫu của bộ dữ liệu Golub

Insight về thiết kế thí nghiệm cần đưa vào báo cáo

Confounding trong tập huấn luyện: toàn bộ ALL là trẻ em và toàn bộ AML là người lớn, lại đến từ hai ngân hàng mẫu khác nhau. Vì vậy một số gen "phân biệt ALL/AML" có thể thực chất phản ánh tuổi hoặc nguồn mẫu.

Batch effect: bài báo ghi nhận prediction strength trung bình thấp hơn ở mẫu của phòng lab dùng quy trình khác. Nhóm sẽ kiểm tra hiện tượng này bằng PCA tô màu theo nguồn mẫu.

Tập kiểm tra khác phân phối (có máu ngoại vi, AML trẻ em) nên là bài kiểm tra khả năng tổng quát hóa tốt hơn là chia ngẫu nhiên.

1.5.2. Thiết kế thí nghiệm tính toán của nhóm

Phân phân tích của nhóm được tổ chức như một **thí nghiệm giai thừa (factorial design)**, trong đó mỗi cấu hình là một tổ hợp các nhân tố. Cách tổ chức này giúp so sánh công bằng và trả lời được câu hỏi "nhân tố nào ảnh hưởng nhiều nhất đến hiệu năng".

Nhân tố	Các mức (levels)
A. Phương pháp chọn biến	SNR của Golub, t-test + FDR, LASSO, Elastic Net, RF importance, SVM-RFE
B. Số gen giữ lại (k)	5, 10, 20, 50, 100, 200
C. Bộ phân loại	Weighted voting (Golub), k-NN, DLDA, Logistic (L1/L2), SVM tuyến tính, Random Forest
D. Cách đánh giá	Tách train/test gốc; LOOCV; Repeated stratified 5-fold; Nested CV; CV "sai" (chọn gen trước CV)
Biến phản hồi	Balanced accuracy, AUC-ROC, số mẫu không chắc chắn, độ ổn định tập gen (Jaccard)

Bảng 1.4. Các nhân tố của thí nghiệm tính toán

Nguyên tắc DoE áp dụng: **randomization** (cố định random seed, báo cáo seed), **replication** (lặp lại CV 10–20 lần để ước lượng phương sai), **blocking** (stratified split để giữ tỷ lệ ALL/AML trong mọi fold), và **control** (so sánh với baseline dự đoán lớp đa số và với mô hình huấn luyện trên nhãn hoán vị ngẫu nhiên).

1.6. Giả thuyết nghiên cứu và tiêu chí thành công

Mã	Giả thuyết	Cách kiểm chứng
H1	Có nhiều gen tương quan với phân biệt ALL/AML hơn mức ngẫu nhiên	Permutation test (neighborhood analysis), t-test + Benjamini–Hochberg FDR
H2	Chỉ cần 10–50 gen là đủ phân loại với balanced accuracy $\geq 95\%$	Đường cong hiệu năng theo k trong nested CV
H3	Chọn gen trước khi CV làm hiệu năng bị thổi phồng rõ rệt	So sánh CV đúng và CV sai trên nhãn thật và nhãn hoán vị
H4	Mô hình đơn giản (DLDA, weighted voting) không thua mô hình phức tạp khi n nhỏ	So sánh các classifier, kiểm định McNemar / khoảng tin cậy
H5	Dữ liệu có cấu trúc cụm tự nhiên trùng với ALL/AML và B/T-ALL	k-means / hierarchical clustering, Adjusted Rand Index
H6	Nguồn mẫu / loại mô tạo batch effect nhận thấy được	PCA tô màu theo nguồn và loại mô, PERMANOVA hoặc kiểm định trên PC

Bảng 1.5. Giả thuyết nghiên cứu

Tiêu chí thành công (KPI) của dự án: (1) mô hình cuối có balanced accuracy $\geq 95\%$ và AUC ≥ 0.97 trong nested CV; (2) tập gen cuối ≤ 50 gen với độ ổn định Jaccard ≥ 0.5 qua bootstrap; (3) toàn bộ kết quả chạy lại được bằng một lệnh; (4) thuyết trình đúng thời gian và mỗi hình trả lời đúng một câu hỏi.

2. Thu thập và quản trị dữ liệu

2.1. Nguồn dữ liệu và cách ghép dữ liệu

2.1.1. Hai nguồn dữ liệu được sử dụng

Nhóm dùng bộ dữ liệu có sẵn, không cào dữ liệu. Toàn bộ xử lý viết bằng **Python**. Dữ liệu được lấy từ hai nguồn công khai, mỗi nguồn đóng một vai trò riêng:

Nguồn	File	Phần được sử dụng	Lý do
Kaggle: "Gene expression dataset (Golub et al.)"	data_set_ALL_AML_train.csv (38 mẫu) data_set_ALL_AML_independent.csv (34 mẫu) actual.csv (nhân)	Toàn bộ dữ liệu biểu hiện gen (7129 probe) và nhân ALL/AML	Là dữ liệu gốc, chưa qua threshold/lọc/log, nên nhóm tự tiền xử lý được (công việc 1.2); giữ nguyên cách chia train/test của bài báo
OpenIntro: bộ dữ liệu "golub"	golub.csv (72 hàng; 6 cột mô tả + 7129 cột biểu hiện)	Chỉ 6 cột mô tả mẫu (metadata)	Kaggle không có thông tin loại mô, nguồn, giới tính, B/T-ALL; 7129 cột biểu hiện của OpenIntro đã được chuẩn hóa nên bỏ đi
Bài báo Golub et al. (1999)	Phần chú thích (notes 13, 15, 23)	Thông tin về tuổi, quy trình lab	Cả hai bộ dữ liệu đều không có tuổi bệnh nhân; nhóm dùng mô tả trong bài báo

Bảng 2.1. Nguồn dữ liệu

Cấu trúc gốc của file Kaggle hơi đặc biệt và cần xử lý trước khi ghép: mỗi hàng là một probe (gen), hai cột đầu là "Gene Description" và "Gene Accession Number", sau đó là các cặp cột xen kẽ gồm cột giá trị (tên cột là mã bệnh nhân, ví dụ "1", "2"...) và cột "call" (P/A/M). Thứ tự các cột bệnh nhân không nhất thiết theo số thứ tự. File actual.csv có hai cột: patient (mã bệnh nhân 1–72) và cancer (ALL hoặc AML).

Sáu cột mô tả của OpenIntro được sử dụng như sau:

Cột OpenIntro	Ý nghĩa	Giá trị gốc	Sau khi chuẩn hóa
Samples	Mã mẫu (số thứ tự bệnh nhân)	Số nguyên 1–72	sample_id (khóa ghép)
BM.PB	Loại mô lấy mẫu	BM (tủy xương), PB (máu ngoại vi)	tissue: BM / PB
Gender	Giới tính	F, M, có thể thiếu (NA)	sex: F / M / NA
Source	Bệnh viện, ngân hàng mẫu nguồn	CALGB, CCG, DFCI, St-Jude	source: giữ nguyên
tissue.mf	Tổ hợp loại mô và giới tính	BM:f, BM:m, BM:NA, PB:f, PB:m, PB:NA	Bỏ (tạo lại được từ tissue và sex); chỉ dùng để kiểm tra chéo
cancer	Loại bệnh chi tiết	allB, allT, aml	subtype: B-ALL / T-ALL / AML

Bảng 2.2. Sáu cột mô tả lấy từ OpenIntro

2.1.2. Quy trình ghép dữ liệu

Nguyên tắc quan trọng nhất: **ghép theo mã mẫu, không bao giờ ghép theo thứ tự dòng**. Vì thứ tự cột trong file Kaggle và thứ tự hàng trong OpenIntro có thể khác nhau, ghép theo vị trí sẽ gán nhầm metadata cho bệnh nhân mà không báo lỗi.

1. **Đọc và định hình lại file Kaggle:** với mỗi file train/independent, bỏ hai cột mô tả và tất cả cột "call", chuyển vị để mỗi hàng là một bệnh nhân, đổi tên cột thành mã probe, gán cột split = train hoặc test.
2. **Gộp train và test** thành một bảng 72 hàng, sắp xếp theo mã bệnh nhân.
3. **Ghép nhãn** từ actual.csv theo mã bệnh nhân (patient = sample_id), đặt tên cột là class.
4. **Đọc OpenIntro chỉ với 6 cột mô tả** (tham số usecols của pandas), đổi tên và mã hóa lại giá trị theo Bảng 2.2.
5. **Ghép metadata** vào bảng mẫu bằng left join theo sample_id, kèm kiểm tra quan hệ một-một (validate="one_to_one").
6. **Kiểm tra sau ghép:** đủ 72 mẫu; không mẫu nào thiếu metadata; class từ Kaggle nhất quán với subtype từ OpenIntro (B-ALL và T-ALL thuộc ALL, AML thuộc AML) cho cả 72 mẫu; phân bố theo split, tissue, source khớp với mô tả của bài báo (Bảng 2.4).
7. **Tách thành hai file:** ma trận biểu hiện (expression.parquet) và bảng mô tả mẫu (samples.parquet), dùng chung chỉ mục sample_id.

Nếu kiểm tra không khớp

Không sửa tay dữ liệu. Liệt kê các mẫu lệch, kiểm tra lại cột Samples của OpenIntro có đúng là mã bệnh nhân không, rồi ghi quyết định vào docs/decisions.md. Nguyên tắc ưu tiên: **nhãn ALL/AML lấy theo Kaggle** (bộ dữ liệu chính), metadata chỉ dùng cho phân tích phụ.

2.1.3. Định dạng dữ liệu sau khi ghép (để kiểm tra)

Sau khi ghép, người thực hiện đối chiếu kết quả với các mẫu dưới đây. **Các giá trị cụ thể trong hai bảng demo chỉ mang tính minh họa định dạng;** số liệu thật lấy từ file. Riêng các con số trong Bảng 2.4 là giá trị kỳ vọng dựa trên bài báo và phải khớp.

File 1: data/standardized/samples.parquet (72 hàng × 6 cột, chỉ mục sample_id):

sample_id	split	class	subtype	tissue	source	sex
1	train	ALL	B-ALL	BM	DFCI	M
2	train	ALL	T-ALL	BM	DFCI	M
...	...	...	...	...	...	...
28	train	AML	AML	BM	CALGB	F
...	...	...	...	...	...	...
50	test	AML	AML	PB	CCG	NA
...	...	...	...	...	...	...

Bảng 2.3a. Demo định dạng bảng mô tả mẫu (giá trị minh họa)

File 2: data/standardized/expression.parquet (72 hàng × 7129 cột, chỉ mục sample_id, tên cột là mã probe, kiểu số nguyên, giá trị chưa xử lý nên có thể âm):

sample_id	AFFX-BioB-5_at	AFFX-BioB-M_a t	...	X95735_at	...	(probe thứ 7129)
1	-214	-153	...	-8	...	-37
2	-139	-73	...	12	...	-18
...	...	...	...	...	...	...

sample_id	AFFX-BioB-5_at	AFFX-BioB-M_at	...	X95735_at	...	(probe thứ 7129)
72	-35	-128	...	1340	...	-40

Bảng 2.3b. Demo định dạng ma trận biểu hiện (giá trị minh họa)

Kết quả kiểm tra kỳ vọng (chạy các lệnh `pd.crosstab` trên `samples.parquet`; các số dưới đây lấy từ bài báo và phải khớp):

Kiểm tra	Kết quả kỳ vọng
Kích thước	expression: (72, 7129); samples: (72, 6)
class	ALL 47, AML 25
split × class	train: 27 ALL, 11 AML; test: 20 ALL, 14 AML
split × tissue	train: 38 BM; test: 24 BM, 10 PB
Nguồn của tập train	ALL: 27 DFCI; AML: 11 CALGB
Nguồn của tập test	ALL: 17 DFCI, 3 St-Jude; AML: 4 CALGB, 5 St-Jude, 5 CCG
subtype	B-ALL và T-ALL cộng lại bằng 47; AML bằng 25; không có giá trị thiếu
Khớp nhãn	class (Kaggle) nhất quán với subtype (OpenIntro) ở 72/72 mẫu

Bảng 2.4. Các kiểm tra bắt buộc sau khi ghép

2.2. Kiến trúc lưu trữ theo tầng

Dữ liệu chỉ khoảng $72 \times 7129 \approx 513$ nghìn giá trị (vài MB), nên không cần HDFS hay Spark. Nhóm áp dụng tư duy **phân tầng của data lake** trong bài giảng để đảm bảo truy vết và tái lập, theo ba nguyên tắc:

- **Dữ liệu chỉ chảy một chiều**: raw → standardized → cleansed → curated. Mỗi tầng được sinh tự động từ tầng trước bằng code.
- **Không ai sửa tay dữ liệu**: nếu phát hiện lỗi, sửa code rồi chạy lại. Nhờ vậy mọi con số trong báo cáo truy ngược được về file gốc.
- **Notebook chỉ đọc từ tầng curated** (hoặc qua hàm `load_golub()`) và chỉ ghi vào `results/`.

```

golub-project/
├── data/
│   ├── raw/ # Tầng 1: dữ liệu gốc, không bao giờ sửa
│   │   ├── kaggle/
│   │   │   ├── data_set_ALL_AML_train.csv
│   │   │   ├── data_set_ALL_AML_independent.csv
│   │   │   └── actual.csv
│   │   ├── openintro/golub.csv
│   │   └── SOURCES.md # link, ngày tải, checksum MD5
│   ├── standardized/ # Tầng 2: cùng cấu trúc, chưa đánh giá chất lượng
│   │   ├── expression.parquet # 72 mẫu × 7129 probe, giá trị gốc
│   │   ├── calls.parquet # 72 × 7129, giá trị P/A/M
│   │   ├── samples.parquet # bảng mô tả mẫu (Dim_Sample)
│   │   └── genes.parquet # bảng mô tả probe (Dim_Gene)
│   ├── cleansed/ # Tầng 3: đã kiểm tra chất lượng
│   │   ├── expression.parquet
│   │   ├── samples.parquet # thêm cột cờ chất lượng
│   │   └── quality_report.md
│   ├── curated/ # Tầng 4: sẵn sàng cho phân tích
│   │   ├── X_log10.parquet # 72 × 7129, đã threshold + log10
│   │   ├── X_log10_filtered.parquet # 72 × ~3571, đã lọc gen (cho EDA)
│   │   ├── samples.parquet
│   │   └── preprocess_params.json
│   └── sandbox/ # Tầng 5: nháp riêng từng người
│       └── tv1/ tv2/ ... tv6/
├── config/
│   └── experiment_config.yaml
├── src/
│   ├── config.py load.py quality.py preprocess.py
│   ├── golub_wv.py features.py models.py evaluate.py viz.py
├── notebooks/
│   ├── 01_eda.ipynb 02_design_batch.ipynb 03_discovery.ipynb
│   ├── 04_de_genes.ipynb 05_golub_reproduction.ipynb 06_selection.ipynb
│   └── 07_models.ipynb 08_evaluation.ipynb 09_interpretation.ipynb
├── results/
│   ├── metrics/ figures/report/ figures/slides/ gene_lists/
├── tests/
│   ├── test_no_leakage.py test_load.py
├── docs/
│   ├── data_dictionary.md style_guide.md decisions.md
│   ├── meeting_notes/ handover/ report/
└── environment.yml Makefile README.md .gitignore

```

2.2.1. Các tầng dữ liệu và ý nghĩa từng file

Tầng	File	Biểu diễn gì	Tạo bởi / Dùng bởi
raw	kaggle/data_set_ALL_AML_train.csv kaggle/data_set_ALL_AML_independent.csv kaggle/actual.csv	Bản gốc tải về, giữ nguyên định dạng Kaggle (probe theo hàng, cột call xen kẽ)	Tải tay một lần / chỉ load.py đọc
raw	openintro/golub.csv	Bản gốc OpenIntro (6 cột mô tả + 7129 cột biểu hiện đã chuẩn hóa)	Tải tay một lần / chỉ load.py đọc 6 cột mô tả
raw	SOURCES.md	Link tải, ngày tải, mã MD5 của từng file; chứng minh dữ liệu gốc không bị thay đổi	TV2
standardized	expression.parquet	Ma trận 72×7129 : mỗi hàng một bệnh nhân, mỗi cột một probe; giá trị gốc, chưa sửa	load.py / quality.py
standardized	calls.parquet	Ma trận 72×7129 các giá trị P/A/M (có mặt/vắng mặt/không rõ); lưu cho đầy đủ, không dùng để mô hình	load.py / tham khảo
standardized	samples.parquet	Bảng mô tả mẫu (Dim_Sample): sample_id, split, class, subtype, tissue, source, sex	load.py / quality.py
standardized	genes.parquet	Bảng mô tả probe (Dim_Gene): probe_id, description, is_control (tiền tố AFFX)	load.py / TV4, TV5 khi diễn giải gen
cleansed	expression.parquet	Giống tầng trước nhưng đã qua kiểm tra chất lượng; giá trị vẫn chưa bị biến đổi	quality.py / preprocess.py
cleansed	samples.parquet	Bảng mô tả mẫu có thêm các cột cờ chất lượng (ví dụ qc_outlier); đánh dấu chứ không xóa	quality.py / mọi người
cleansed	quality_report.md	Báo cáo kiểm tra: số giá trị âm, bão hòa, nhân khớp, mẫu bất thường	quality.py / TV1 nghiệm thu
curated	X_log10.parquet	72×7129 sau threshold [100, 16000] và log10 (phép biến đổi cố định, an toàn cho cả 72 mẫu). Đầu vào của mọi mô hình; lọc gen và chuẩn hóa làm trong Pipeline	preprocess.py / TV1, TV2, TV3, TV4
curated	X_log10_filtered.parquet	$72 \times$ khoảng 3571 gen sau khi lọc thêm theo Dudoit. Chỉ dùng cho EDA, phân cụm, gen khác biệt, không dùng để đánh giá mô hình	preprocess.py / TV4, TV5, TV6, TV1
curated	samples.parquet	Bảng mô tả mẫu bản cuối, cùng thứ tự với hai ma trận trên	preprocess.py / mọi người
curated	preprocess_params.json	Ngưỡng, tiêu chí lọc, số gen còn lại; để báo cáo trích dẫn chính xác	preprocess.py / báo cáo

Tầng	File	Biểu diễn gì	Tạo bởi / Dùng bởi
sandbox	tv1/ ... tv6/	Không gian nháp của từng người; không kết quả nào trong báo cáo được lấy từ đây	Mỗi thành viên

Bảng 2.5. Ý nghĩa từng file trong các tầng dữ liệu

2.2.2. Các thư mục khác

Thư mục / file	Biểu diễn gì
config/experiment_config.yaml	Cấu hình thí nghiệm giai thừa: phương pháp chọn biến, số gen, classifier, sơ đồ CV, seed. Code đọc từ đây thay vì ghi cứng tham số
src/config.py	Đường dẫn các thư mục, random seed mặc định, bảng màu ALL/AML dùng chung
src/load.py	Đọc raw, ghép dữ liệu (mục 2.1.2), ghi standardized; hàm load_golub() trả về X và bảng mẫu cho cả nhóm
src/quality.py	Các hàm kiểm tra chất lượng, ghi cleansed và quality_report.md
src/preprocess.py	Threshold, log10, ghi curated; các transformer lọc gen và chuẩn hóa theo chuẩn scikit-learn
src/golub_wv.py	Classifier weighted voting và prediction strength của Golub
src/features.py	Các bộ chọn gen: SNR, t-test, LASSO, Elastic Net, RF importance, SVM-RFE
src/models.py	Định nghĩa các Pipeline mô hình và lưới siêu tham số
src/evaluate.py	Các sơ đồ CV, chỉ số, khoảng tin cậy, kiểm định, thí nghiệm selection bias
src/viz.py	Các hàm vẽ theo style guide
notebooks/	01_eda, 02_design_batch, 03_discovery, 04_de_genes, 05_golub_reproduction, 06_selection, 07_models, 08_evaluation, 09_interpretation; mỗi notebook ứng với một công việc ở chương 5
results/metrics/runs.parquet	Bảng Dim_Run: mỗi hàng một cấu hình thí nghiệm kèm các chỉ số; mọi biểu đồ so sánh đọc từ đây
results/metrics/summary_*.csv	Các bảng tổng hợp đưa thẳng vào báo cáo
results/figures/report/, slides/	Hình F1–F12: bản đồ phân giải cao cho báo cáo và bản chữ to cho slide
results/gene_lists/	gene_ranking.csv (1.6), selected_<phương pháp>.csv (2.2), final_genes.csv (2.2), gene_annotations.csv (2.5)
tests/	test_load.py (kiểm tra Bảng 2.4), test_no_leakage.py (tiền xử lý không nhìn thấy dữ liệu test)
docs/	data_dictionary.md, style_guide.md, decisions.md (nhật ký quyết định), meeting_notes/, handover/, report/
environment.yml, Makefile, README.md	Môi trường Python cố định phiên bản; các lệnh make data, make experiments, make figures, make all (hoặc run_all.py nếu dùng Windows); hướng dẫn chạy

Bảng 2.6. Các thư mục và file còn lại

Thư mục data/ được đưa vào .gitignore (trừ SOURCES.md) vì có thể tạo lại bằng lệnh make data; chỉ mã nguồn, cấu hình và tài liệu được đưa lên GitHub.

2.3. Mô hình dữ liệu

Về mặt logic, dữ liệu được tổ chức theo **lược đồ hình sao** trong bài giảng OLAP. Về mặt vật lý, nhóm lưu dạng ma trận rộng vì thuận tiện cho học máy; khi cần truy vấn kiểu OLAP, ma trận được chuyển sang dạng dài bằng pandas.melt hoặc truy vấn trực tiếp file Parquet bằng DuckDB.

Bảng logic	Loại	File vật lý	Thuộc tính chính
Fact_Expression	Bảng sự kiện	expression.parquet (dạng rộng), calls.parquet	sample_id, probe_id, expression_value, call
Dim_Sample	Bảng chiều	samples.parquet	sample_id, split, class, subtype, tissue, source, sex, cờ chất lượng
Dim_Gene	Bảng chiều	genes.parquet	probe_id, description, is_control
Dim_Run	Bảng kết quả	results/metrics/runs.parquet	run_id, selection, k, classifier, cv_scheme, seed, các chỉ số

Bảng 2.7. Lược đồ hình sao và file vật lý tương ứng

Ví dụ thao tác OLAP: slice (chỉ mẫu tủy xương), dice (AML của tập test từ St-Jude), roll-up (trung bình biểu hiện một gen theo class và source).

2.4. Quy trình xử lý dữ liệu (pipeline)

Bước	Lệnh	Script	Đầu vào → Đầu ra
1. Nạp và ghép	make standardize	src/load.py	raw/ → standardized/
2. Kiểm tra chất lượng	make quality	src/quality.py	standardized/ → cleansed/ + quality_report.md
3. Tiền xử lý	make curate	src/preprocess.py	cleansed/ → curated/
4. Kiểm thử	make test	pytest tests/	Kiểm tra Bảng 2.4 và chống rò rỉ
Cả chuỗi	make data	Chạy bước 1–4	raw/ → curated/

Bảng 2.8. Pipeline dữ liệu

2.5. Kiểm tra chất lượng dữ liệu

Áp dụng phân loại vấn đề chất lượng dữ liệu trong bài giảng (value, value-set, record, relation):

Mức	Vấn đề cần kiểm tra	Cách kiểm tra	Xử lý
Value	Giá trị thiếu; giá trị âm hoặc rất nhỏ (đặc thù thuật toán MAS4 của Affymetrix); giá trị bão hòa rất lớn	Đếm NaN, đếm giá trị < 100 và > 16000	Threshold về khoảng [100, 16000] ở bước tiền xử lý
Value-set	Probe điều khiển (AFFX-...) lẫn với probe gen; nhiều probe cho cùng một gen; giá trị metadata ngoài danh mục	Lọc theo tiền tố; đếm trùng mô tả; kiểm tra giá trị của tissue, source, subtype	Đánh dấu is_control; ghi chú khi diễn giải gen
Record	Mẫu trùng lặp; mẫu có phân phối bất thường (chip lỗi)	Tương quan giữa các mẫu; boxplot theo mẫu	Gắn cờ qc_outlier; không xóa nếu không có căn cứ
Relation	Nhãn Kaggle và subtype OpenIntro không nhất quán; ghép sai mẫu	Các kiểm tra ở Bảng 2.4	Ưu tiên nhãn Kaggle; ghi vào decisions.md

Bảng 2.9. Kế hoạch kiểm tra chất lượng dữ liệu

2.6. Tiền xử lý và biểu diễn dữ liệu

Nhóm áp dụng quy trình tiền xử lý được dùng rộng rãi cho bộ dữ liệu này (Dudoit, Fridlyand & Speed, 2002):

1. **Thresholding:** giá trị < 100 gán bằng 100, giá trị > 16000 gán bằng 16000.
2. **Log-transform:** lấy $\log_{10}$ để giảm độ lệch phải và ổn định phương sai. Hai bước 1–2 là phép biến đổi cố định, cho ra `X_log10.parquet`.
3. **Filtering:** giữ gen có $\max/\min > 5$ và $(\max - \min) > 500$ (tính trên thang gốc); kỳ vọng còn khoảng 3571 gen khi lọc trên 72 mẫu, cho ra `X_log10_filtered.parquet` dùng cho khám phá.
4. **Standardization:** chuẩn hóa mỗi gen về trung bình 0, độ lệch chuẩn 1 (tùy mô hình).

Nguyên tắc chống rò rỉ dữ liệu (data leakage)

Khi xây dựng và đánh giá mô hình, bước lọc gen, chuẩn hóa và chọn gen phải nằm trong **scikit-learn Pipeline** và chỉ fit trên phần huấn luyện của từng fold. Vì vậy mô hình đọc `X_log10.parquet` (chưa lọc), không đọc bản đã lọc.

2.7. Quản trị dữ liệu và tính tái lập

- **Data dictionary:** mô tả mọi cột, kiểu dữ liệu, đơn vị, nguồn gốc và phép biến đổi đã áp dụng.
- **Quản lý phiên bản:** mã nguồn trên GitHub (nhánh main được bảo vệ, merge qua pull request có review); dữ liệu không đưa lên Git, chỉ lưu checksum trong `SOURCES.md`.
- **Tái lập:** `environment.yml` cố định phiên bản Python và thư viện (pandas, pyarrow, numpy, scipy, statsmodels, scikit-learn, matplotlib, seaborn, umap-learn, minisom, pytest); lệnh `make all` chạy từ raw đến hình; mọi random seed được ghi trong `Dim_Run`.
- **Đạo đức và pháp lý:** dữ liệu đã được ẩn danh và công bố công khai; nhóm trích dẫn đầy đủ nguồn (Kaggle, OpenIntro, bài báo gốc) và không cố gắng định danh bệnh nhân.

3. Phân tích, khám phá và tìm kiếm insight

3.1. Phân tích khám phá (EDA)

3.1.1. Đơn biến và theo mẫu

- Tỷ lệ lớp trong train, test và toàn bộ; phân bố theo loại mô, nguồn, giới tính.
- Histogram và density của giá trị biểu hiện trước và sau log10; tính skewness và kurtosis để minh chứng lý do cần log-transform.
- Boxplot phân bố biểu hiện theo từng mẫu (72 hộp), tô màu theo nguồn: phát hiện chip lỗi và khác biệt giữa các phòng lab.
- Phân bố phương sai và hệ số biến thiên của các gen: minh họa rằng phần lớn gen gần như không đổi và giải thích bước lọc.

3.1.2. Đa biến và giảm chiều

- **PCA**: scree plot (phương sai giải thích), biểu đồ PC1–PC2 tô màu theo ALL/AML, rồi lặp lại tô màu theo nguồn và loại mô để phát hiện batch effect (H6).
- **t-SNE / UMAP**: so sánh với PCA; lưu ý khoảng cách giữa các cụm trên t-SNE không có ý nghĩa định lượng.
- **Ma trận tương quan** giữa các mẫu và heatmap có hierarchical clustering (khoảng cách 1 – Pearson, liên kết average).

3.2. Class discovery (khám phá lớp không giám sát)

Tái lập ý tưởng của Golub: nếu chưa biết ALL/AML, liệu dữ liệu có tự "phát hiện" ra hai loại bệnh không? Nhóm dùng k-means và hierarchical clustering (có thể thêm SOM qua thư viện MiniSom) với $k = 2$ và $k = 4$, rồi so sánh cụm với nhãn thật bằng **Adjusted Rand Index** và bảng chéo. Bài báo gốc báo cáo SOM 2 cụm gần khớp ALL/AML (24/25 ALL ở cụm A1, 10/13 AML ở cụm A2) và SOM 4 cụm tách được AML, T-ALL và B-ALL.

3.3. Chọn biến (variable selection)

Nhóm	Phương pháp	Ý tưởng	Ưu / nhược
Filter	Signal-to-noise của Golub: $P(g,c) = (\mu_1 - \mu_2)/(\sigma_1 + \sigma_2)$	Xếp hạng từng gen theo khả năng tách hai lớp	Nhanh, dễ giải thích / bỏ qua tương quan giữa các gen
Filter	Welch t-test + Benjamini–Hochberg FDR	Kiểm định khác biệt trung bình, kiểm soát tỷ lệ phát hiện sai	Có cơ sở thống kê / giả định gần chuẩn
Filter	Neighborhood analysis (permutation test)	So số gen tương quan với nhãn thật và với nhãn hoán vị	Trả lời H1 trực tiếp / tốn thời gian tính toán
Embedded	LASSO (L1) và Elastic Net	Hồi quy logistic có phạt, đẩy hệ số không quan trọng về 0	Tự chọn số gen / LASSO chọn tối đa n gen, bất ổn khi gen tương quan cao
Embedded	Random Forest importance	Mức giảm impurity hoặc permutation importance	Bắt quan hệ phi tuyến / thiên lệch với biến tương quan
Wrapper	SVM-RFE (Guyon et al., 2002)	Lặp lại huấn luyện SVM và loại gen có trọng số nhỏ nhất	Thường hiệu quả / tốn tài nguyên, dễ overfit

Bảng 3.1. Các phương pháp chọn biến sẽ so sánh

Ngoài hiệu năng dự đoán, nhóm đánh giá **độ ổn định** của tập gen: chạy chọn biến trên 100 mẫu bootstrap, tính tần suất mỗi gen được chọn và chỉ số Jaccard trung bình giữa các lần. Nhóm cũng so sánh mức trùng lặp giữa các phương pháp (biểu đồ UpSet) và với danh sách 50 gen của Golub.

3.4. Phân loại và dự đoán

Mô hình	Lý do chọn	Siêu tham số cần tinh chỉnh
Weighted voting (Golub)	Tái lập baseline gốc; có cơ chế "không chắc chắn" qua prediction strength	Số gen; ngưỡng PS
Diagonal LDA / Gaussian Naive Bayes	Rất ít tham số, thường mạnh khi n nhỏ (Dudoit et al., 2002)	Số gen
k-NN	Không tham số, dễ giải thích	k , metric (Euclid, tương quan)
Logistic regression L1/L2	Kết hợp chọn biến và phân loại, cho xác suất	C (độ mạnh regularization), $l1_ratio$
SVM tuyến tính	Hoạt động tốt khi $p \gg n$	C
Random Forest	Mô hình phi tuyến, đại diện cho phương pháp ensemble	$n_estimators$, $max_features$

Bảng 3.2. Các bộ phân loại

3.5. Đánh giá mô hình (evaluation)

3.5.1. Các sơ đồ đánh giá

1. **Tách gốc 38/34**: huấn luyện trên 38 mẫu, kiểm tra trên 34 mẫu độc lập, giống bài báo. Lưu ý: với 34 mẫu test, mỗi lỗi tương đương khoảng 3 điểm phần trăm accuracy, nên cần báo cáo khoảng tin cậy.
2. **LOOCV đúng cách** trên 38 mẫu train, trong đó chọn gen được lặp lại từ đầu cho mỗi fold, đúng như note 22 của bài báo.
3. **Nested repeated stratified CV** trên toàn bộ 72 mẫu: vòng trong chọn k và siêu tham số, vòng ngoài ước lượng hiệu năng; lặp lại 10–20 lần.
4. **CV sai cách (để minh họa)**: chọn gen trên toàn bộ dữ liệu rồi mới CV.

3.5.2. Thí nghiệm then chốt: selection bias

Thí nghiệm này là điểm nhấn của project. Nhóm hoán vị ngẫu nhiên nhãn ALL/AML (xóa mọi quan hệ thật giữa gen và nhãn), rồi chạy cả CV đúng và CV sai. Với nhãn ngẫu nhiên, hiệu năng thật phải quanh 50% (hoặc quanh tỷ lệ lớp đa số). Nếu CV sai cho kết quả cao hơn nhiều, đó là bằng chứng trực quan cho selection bias (H3). Lặp lại 100 lần để có phân phối.

```
pipe = Pipeline([
    ('scale', StandardScaler()),
    ('select', SelectKBest(score_func=f_classif)), # chọn gen NẪM TRONG pipeline
    ('clf', LogisticRegression(penalty='l2', max_iter=5000)),
])
grid = {'select__k': [10, 20, 50, 100], 'clf__C': [0.01, 0.1, 1, 10]}
inner = StratifiedKFold(5, shuffle=True, random_state=1)
outer = RepeatedStratifiedKFold(n_splits=5, n_repeats=10, random_state=2)
search = GridSearchCV(pipe, grid, cv=inner, scoring='balanced_accuracy')
scores = cross_val_score(search, X, y, cv=outer, scoring='balanced_accuracy')
```

3.5.3. Chỉ số và kiểm định

- **Balanced accuracy** (chỉ số chính vì lớp mất cân bằng), AUC-ROC, sensitivity/specificity (coi AML là lớp dương), F1, ma trận nhầm lẫn.
- **Khoảng tin cậy**: Wilson interval cho accuracy trên tập test; trung bình $\pm$ độ lệch chuẩn qua các lần lặp CV.
- **So sánh mô hình**: kiểm định McNemar trên tập test; so sánh phân phối điểm qua các lần lặp CV (lưu ý các fold không độc lập nên diễn giải thận trọng).
- **Calibration và từ chối dự đoán**: tỷ lệ mẫu có độ tin cậy thấp ($PS < 0.3$ hoặc xác suất trong khoảng 0.35–0.65), để nói lại với nhu cầu "không chắc chắn" của bác sĩ.
- **Phân tích lỗi**: các mẫu bị dự đoán sai thuộc nguồn nào, loại mô nào (liên hệ với batch effect).

4. Trực quan hóa dữ liệu, kết quả và storytelling

4.1. Nguyên tắc thiết kế (áp dụng từ bài giảng)

Nguyên tắc	Áp dụng cụ thể trong project
Expressiveness & Effectiveness (Mackinlay)	Dữ liệu định lượng quan trọng nhất (accuracy, AUC) mã hóa bằng vị trí trên trục chung; không dùng biểu đồ tròn hay 3D
Xếp hạng cảm nhận (Cleveland & McGill)	Ưu tiên dot plot và bar chart hơn diện tích hay màu khi cần so sánh số liệu chính xác
Thang màu định tính	ALL/AML dùng cặp màu an toàn với người mù màu (xanh dương / cam), nhất quán trong mọi hình
Thang màu phân kỳ	Heatmap z-score dùng thang xanh–trắng–đỏ, tâm tại 0; không dùng thang cầu vồng (rainbow)
Mã hóa dư thừa (redundant coding)	Kết hợp màu với hình dạng điểm (tròn/tam giác) để in đen trắng vẫn đọc được
Proportional ink	Bar chart luôn bắt đầu từ 0; nếu muốn phóng to vùng 0.9–1.0 thì dùng dot plot
Xử lý chồng lấn điểm	Độ trong suốt (alpha) và jitter nhẹ trong strip plot; 2D density khi nhiều điểm
Small multiples	So sánh 6 phương pháp chọn biến bằng các panel cùng trục thay vì chồng 6 đường
Highlight và direct labeling	Tô nổi bật mô hình cuối cùng, làm mờ phần còn lại; ghi nhãn trực tiếp thay cho chú giải khi có thể
Tiền chú ý (pre-attentive)	Dùng một màu nổi bật duy nhất cho thông điệp chính của mỗi slide

Bảng 4.1. Nguyên tắc trực quan hóa và cách áp dụng

Nhóm sẽ viết một **style guide** ngắn (docs/style_guide.md) gồm bảng màu, font, cỡ chữ trục (tối thiểu 10–12 pt khi in), kích thước hình và mẫu tiêu đề, cùng một module viz.py dùng chung để mọi thành viên tạo hình thống nhất.

4.2. Danh mục biểu đồ dự kiến

#	Biểu đồ	Câu hỏi trả lời	Mã hóa trực quan
F1	Bar chart ngang: số mẫu theo lớp × nguồn × loại mô	Thiết kế mẫu có cân bằng không?	Độ dài (Q), màu định tính (N), small multiples theo train/test
F2	Histogram trước/sau log10	Vì sao cần log-transform?	Vị trí, 2 panel
F3	Boxplot theo từng mẫu	Có chip bất thường hay batch effect?	Vị trí; màu theo nguồn
F4	Scree plot + PCA scatter (2 phiên bản màu)	Có cấu trúc lớp? Có batch effect?	Vị trí x/y; màu + hình theo lớp; panel 2 tô theo nguồn
F5	Heatmap 50 gen hàng đầu + dendrogram	Các gen phân biệt lớp thế nào?	Thang màu phân kỳ z-score, thanh màu lớp phía trên
F6	Neighborhood analysis / volcano plot	Có nhiều gen tương quan hơn ngẫu nhiên? (H1)	Đường thật so với dải 1%–5% của nhân hoán vị
F7	UpSet plot hoặc heatmap Jaccard	Các phương pháp chọn gen có đồng thuận?	Vị trí, độ dài
F8	Line chart hiệu năng theo k (small multiples)	Cần bao nhiêu gen? (H2)	Vị trí; dải sai số thể hiện độ bất định
F9	Dot plot so sánh classifier kèm khoảng tin cậy	Mô hình nào tốt nhất? (H4)	Vị trí trên trục chung, mô hình tốt nhất được highlight

#	Biểu đồ	Câu hỏi trả lời	Mã hóa trực quan
F10	Phân phối accuracy: CV đúng vs CV sai, nhãn thật vs nhãn hoán vị	Selection bias nghiêm trọng đến đâu? (H3)	Violin/strip plot, đường tham chiếu 50%
F11	ROC curve và ma trận nhầm lẫn	Mô hình cuối hoạt động ra sao trên test?	Vị trí; số đếm ghi trực tiếp
F12	Strip plot prediction strength / xác suất theo mẫu	Mẫu nào không chắc chắn, vì sao?	Vị trí; màu theo nguồn; đường ngưỡng

Bảng 4.2. Danh mục hình dự kiến

5. Phân công công việc và tiến độ

5.1. Bảng phân công công việc kỹ thuật

Nhóm điền tên thật vào cột đầu tiên.

Thành viên	Giai đoạn 1 (tuần 1–4)	Giai đoạn 2 (tuần 5–8)	Mạch liên kết giữa hai việc
TV1	1.3. Phân tích thiết kế mẫu và batch effect	2.1. Tái lập phương pháp của Golub (weighted voting)	Hiểu cách Golub lấy mẫu, rồi kiểm chứng chính kết quả của bài báo; cả hai là xương sống của câu chuyện thuyết trình
TV2	1.1. Nạp, ghép dữ liệu và kiểm tra chất lượng	2.4. Khung đánh giá và thí nghiệm selection bias	Người dựng hạ tầng dữ liệu dựng tiếp hạ tầng đánh giá dùng chung
TV3	1.2. Tiền xử lý và pipeline chống rò rỉ	2.3. Các mô hình phân loại và thí nghiệm giai thừa	Transformer tiền xử lý được dùng trực tiếp trong Pipeline mô hình
TV4	1.6. Phân tích gen khác biệt (H1)	2.2. Chọn biến và độ ổn định	Xếp hạng gen bằng kiểm định là nền cho chọn biến
TV5	1.5. Class discovery (phân cụm)	2.5. Diễn giải sinh học và phân tích lỗi	Người phát hiện các nhóm con giải thích gen và mẫu dự đoán sai
TV6	1.4. EDA, giảm chiều và style guide	2.6. Trực quan hóa kết quả và dashboard	Người lập style guide hoàn thiện toàn bộ hình kết quả

Bảng 5.2. Mỗi thành viên một công việc kỹ thuật trong mỗi giai đoạn

5.2. Mô tả công việc giai đoạn 1: Hiểu dữ liệu

Mục tiêu của giai đoạn 1 là hiểu thật kỹ bộ dữ liệu Golub trước khi xây dựng mô hình. Trong phần demo, **các con số lấy từ bài báo hoặc từ cấu trúc dữ liệu là giá trị bắt buộc phải khớp**; các chỗ ghi "..." là kết quả thực tế người thực hiện điền vào khi nghiệm thu.

1.1. Nạp, ghép dữ liệu và kiểm tra chất lượng (deadline 10/10)

Phụ trách: TV2 **Thời gian:**

Hạng mục	Mô tả
Mục tiêu	Đưa bộ dữ liệu Golub từ Kaggle và metadata OpenIntro vào dạng dùng được cho cả nhóm, đáng tin cậy và tái lập được.
Công việc cụ thể	- Tải 3 file Kaggle và golub.csv của OpenIntro vào data/raw/, ghi SOURCES.md kèm MD5. - Đựng repo GitHub, cấu trúc thư mục (mục 2.2), environment.yml, Makefile, src/config.py. - Viết src/load.py theo quy trình ghép ở mục 2.1.2; tạo expression.parquet, calls.parquet, samples.parquet, genes.parquet. - Viết src/quality.py kiểm tra 4 mức (mục 2.5), ghi tầng cleansed và quality_report.md. - Viết tests/test_load.py cho toàn bộ Bảng 2.4; viết data_dictionary.md.
Phụ thuộc / phối hợp	Không phụ thuộc ai. Là đầu vào của mọi công việc khác.
Sản phẩm bàn giao	src/load.py, src/quality.py, tests/test_load.py; data/standardized/, data/cleansed/; docs/data_dictionary.md; mục 2.1–2.5, 2.7 của báo cáo.

Demo kết quả để nghiệm thu

Người thực hiện chạy các lệnh sau trước trưởng nhóm; kết quả phải khớp như dưới:

```
$ make data && pytest tests/test_load.py
tests/test_load.py ..... [100%] 8 passed

>>> from src.load import load_golub
>>> X, samples = load_golub(layer='cleansed')
>>> X.shape, samples.shape
((72, 7129), (72, 7))      # 6 cột mô tả + cột cờ qc_outlier
>>> samples['class'].value_counts()
ALL    47
AML     25
>>> pd.crosstab(samples['split'], samples['class'])
class ALL AML
split
test    20  14
train   27  11
>>> pd.crosstab([samples['split'], samples['class']], samples['source'])
source  CALGB CCG DFCI St-Jude
split class
test ALL    0  0  17   3
      AML    4  5  0   5
train ALL    0  0  27   0
      AML   11  0  0   0

Trích quality_report.md:
[PASS] 72 mẫu, 7129 probe, không trùng sample_id
[PASS] Nhận Kaggle khớp subtype OpenIntro: 72/72
[INFO] Số giá trị < 100: ... Số giá trị > 16000: ... Probe AFX: ...
[INFO] Mẫu gần cờ qc_outlier: ... (lý do: ...)
```

1.2. Tiền xử lý và pipeline chống rò rỉ

Phụ trách: TV3 **Thời gian:** Tuần 2–3 (chuẩn bị từ tuần 1)

Hạng mục	Mô tả
Mục tiêu	Xây dựng bước tiền xử lý chuẩn cho dữ liệu microarray Golub và đảm bảo không rò rỉ dữ liệu.
Công việc cụ thể	- Cài đặt threshold [100, 16000] và log10; ghi X_log10.parquet. - Cài đặt bộ lọc gen theo Dudoit ($\max/\min > 5$, $\max - \min > 500$); ghi X_log10_filtered.parquet. - Viết transformer GeneFilter và StandardScaler theo chuẩn scikit-learn (fit chỉ trên dữ liệu huấn luyện). - Viết tests/test_no_leakage.py: fit trên 38 mẫu train, kiểm tra tham số không đổi khi thay dữ liệu test. - Ghi preprocess_params.json; viết mục 2.6 của báo cáo.
Phụ thuộc / phối hợp	Nhận load_golub() bản tạm từ TV2 (CN 11/10); giao curated bản 1 cho TV4, TV5, TV6 (CN 18/10).
Sản phẩm bàn giao	src/preprocess.py; tests/test_no_leakage.py; data/curated/; mục 2.6 của báo cáo.

Demo kết quả để nghiệm thu

```
>>> Xl = pd.read_parquet('data/curated/X_log10.parquet')
>>> Xl.shape, round(Xl.min().min(), 3), round(Xl.max().max(), 3)
((72, 7129), 2.0, 4.204)      # log10(100) = 2; log10(16000) ≈ 4.204
>>> Xf = pd.read_parquet('data/curated/X_log10_filtered.parquet')
>>> Xf.shape
(72, 3571)      # nếu lệch vài gen: ghi lý do vào decisions.md

>>> from src.preprocess import GeneFilter
>>> gf = GeneFilter().fit(X_train) # 38 mẫu train
>>> gf.transform(X_test).shape
(34, ...)      # số gen còn lại khi lọc trên 38 mẫu train

$ pytest tests/test_no_leakage.py ... 3 passed
```

1.3. Phân tích thiết kế mẫu và batch effect

Phụ trách: TV1 (trưởng nhóm) **Thời gian:** Tuần 3–4 (chuẩn bị từ tuần 1)

Hạng mục	Mô tả
Mục tiêu	Hiểu mẫu Golub được thu thập thế nào, phát hiện yếu tố gây nhiễu và thiết kế thí nghiệm tính toán cho giai đoạn 2.
Công việc cụ thể	- Lập bảng chéo class × tissue × source cho train và test; vẽ hình F1. - Phân tích confounding: ALL train đều là trẻ em từ DFCI, AML train đều là người lớn từ CALGB (thông tin tuổi lấy từ bài báo). - Kiểm tra batch effect: PCA tô màu theo source và tissue; kiểm định PC1–PC3 theo source (Kruskal–Wallis); so sánh mẫu St-Jude với phần còn lại. - Viết config/experiment_config.yaml cho thí nghiệm giai thừa.
Phụ thuộc / phối hợp	Nhận samples.parquet chính thức từ TV2 (CN 18/10) và curated từ TV3; thống nhất hình PCA với TV6.
Sản phẩm bàn giao	Notebook 02_design_batch; hình F1 và hình batch effect; experiment_config.yaml; mục 1.5 của báo cáo; kết luận H6.

Demo kết quả để nghiệm thu

```
>>> pd.crosstab([samples['split'], samples['tissue']], samples['class'])
class      ALL  AML
split tissue
test BM      ...   ...   # tổng BM của test = 24
    PB      ...   ...   # tổng PB của test = 10
train BM    27  11
```

Kiểm định batch effect (Kruskal–Wallis theo source):
 PC1: $p = \dots$ PC2: $p = \dots$ PC3: $p = \dots$
 Kết luận H6: $\dots$ (có/không có dấu hiệu batch effect rõ ràng; ở PC nào)

```
config/experiment_config.yaml:
selection: [snr, ttest, lasso, enet, rf, svm_rfe]
k: [5, 10, 20, 50, 100, 200]
classifiers: [golub_wv, dlda, knn, logreg_l1, logreg_l2, svm_linear, rf]
cv: {outer_folds: 5, outer_repeats: 10, inner_folds: 5, stratified: true}
seeds: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

1.4. EDA, giảm chiều và style guide

Phụ trách: TV6 **Thời gian:** Tuần 1–4

Hạng mục	Mô tả
Mục tiêu	Hiểu cấu trúc dữ liệu bằng hình ảnh và thống nhất phong cách hình vẽ cho cả dự án.
Công việc cụ thể	- Tuần 1–2: viết style_guide.md (màu ALL/AML an toàn với người mù màu, thang màu phân kỳ cho heatmap, font, cỡ chữ, kích thước hình) và src/viz.py. - EDA đơn biến: histogram trước và sau log10 kèm skewness (F2); boxplot theo 72 mẫu tô màu theo source (F3); phân bố phương sai gen. - Giảm chiều: scree plot, PCA PC1–PC2 tô theo class (F4); t-SNE và UMAP; ma trận tương quan giữa các mẫu. - Viết mục 3.1 của báo cáo.
Phụ thuộc / phối hợp	Nhận curated bản 1 từ TV3 (CN 18/10); giao viz.py cho mọi thành viên (T4 14/10).
Sản phẩm bàn giao	docs/style_guide.md, src/viz.py; notebook 01_eda; hình F2–F4; mục 3.1 của báo cáo.

Demo kết quả thử nghiệm thu

```
>>> from src.viz import pca_scatter
>>> pca_scatter(Xf, samples, color='class', shape='class') # màu + hình dạng
```

results/figures/report/
 F02_hist_before_after_log.png F03_box_per_sample.png
 F04a_pca_scree.png F04b_pca_by_class.png
 F04c_tsne_umap.png

Nhận xét bắt buộc trong notebook:
 Skewness trung vị của các gen: trước log = ..., sau log = ...
 PC1 giải thích ...% phương sai; PC1–PC2 tách ALL/AML: ... (tốt/một phần/không)
 Mẫu có vẻ bất thường trên boxplot/PCA: ... (đối chiếu với cờ qc_outlier)

1.5. Class discovery (phân cụm không giám sát)

Phụ trách: TV5 **Thời gian:** Tuần 3–4 (chuẩn bị từ tuần 1)

Hạng mục	Mô tả
Mục tiêu	Kiểm chứng liệu dữ liệu tự tách thành ALL/AML và B-ALL/T-ALL mà không cần nhãn (H5), như Golub làm bằng SOM.
Công việc cụ thể	- Phân cụm k-means và hierarchical clustering (khoảng cách 1 – Pearson, liên kết average) với $k = 2$ và $k = 4$ trên $X_{\log10_filtered}$. (Tùy chọn) SOM bằng thư viện minisom. - So sánh cụm với class và subtype bằng bảng chéo và Adjusted Rand Index (ARI). - Kiểm tra độ ổn định (10 seed khác nhau) và silhouette; heatmap có dendrogram (F5). - Đối chiếu với bài báo: SOM 2 cụm có 24/25 ALL ở cụm A1 và 10/13 AML ở cụm A2.
Phụ thuộc / phối hợp	Nhận curated bản 1 từ TV3; dùng viz.py của TV6.
Sản phẩm bàn giao	Notebook 03_discovery; hình F5; bảng ARI; mục 3.2 của báo cáo; kết luận H5.

Demo kết quả để nghiệm thu

```
Bảng ARI (điền kết quả thực tế):
method    k  ARI_vs_class  ARI_vs_subtype  silhouette
kmeans    2  ...           -           ...
hierarchical 2  ...           -           ...
kmeans    4  -           ...           ...
hierarchical 4  -           ...           ...

>>> pd.crosstab(clusters_k2, samples['class'])
class    ALL  AML
cluster
0         ...  ...
1         ...  ...

Kết luận H5: ... (so sánh với 24/25 và 10/13 của bài báo)
```

1.6. Phân tích gen khác biệt (H1)

Phụ trách: TV4 **Thời gian:** Tuần 3–4 (chuẩn bị từ tuần 1)

Hạng mục	Mô tả
Mục tiêu	Xác định có bao nhiêu gen thật sự khác biệt giữa ALL và AML và xếp hạng chúng.
Công việc cụ thể	- Tính SNR $P(g,c) = (\mu_1 - \mu_2)/(\sigma_1 + \sigma_2)$ cho mọi gen trên 38 mẫu train. - Neighborhood analysis: hoán vị nhãn khoảng 400 lần, so số gen vượt ngưỡng với dải 1% và 5% (F6); đối chiếu với con số khoảng 1100 gen trong bài báo. - Welch t-test, hiệu chỉnh Benjamini–Hochberg (statsmodels); volcano plot. - Xuất gene_ranking.csv; so sánh top 50 với danh sách 50 gen của Golub.
Phụ thuộc / phối hợp	Nhận curated bản 1 từ TV3; gene_ranking.csv được dùng ở 2.1, 2.2, 2.5.
Sản phẩm bàn giao	Notebook 04_de_genes; hình F6 và volcano plot; results/gene_lists/gene_ranking.csv; kết luận H1.

Demo kết quả để nghiệm thu

```
>>> rank = pd.read_csv('results/gene_lists/gene_ranking.csv')
>>> rank.columns.tolist()
['probe_id', 'description', 'snr', 't_stat', 'p_value', 'q_value', 'rank']
>>> rank.shape[0]
... # bằng số gen sau lọc (khoảng 3571)

Tóm tắt bắt buộc:
Số gen có  $q < 0.05$ : ...
Neighborhood analysis: số gen vượt ngưỡng ở nhân thật = ...;
ngưỡng 1% của nhãn hoán vị = ... → kết luận H1: ...
Số gen trùng giữa top 50 của nhóm và 50 gen của Golub: ... / 50
```

Buổi bàn giao cuối giai đoạn 1 (Chủ nhật 01/11)

Mỗi thành viên trình bày 10 phút và chạy demo nghiệm thu của mình; nộp ghi chú bàn giao (docs/handover/<mã công việc>.md) gồm: đã làm gì, file ở đâu, phát hiện gì, còn vấn đề gì.

Trưởng nhóm tổng hợp phát hiện, rà soát giả thuyết và KPI, chốt kế hoạch chi tiết giai đoạn 2.

5.3. Mô tả công việc giai đoạn 2: Mô hình và kết quả

Tuần 5–6 chạy thí nghiệm, tuần 7 hoàn thiện hình và viết báo cáo, tuần 8 review và tập thuyết trình. Trong phần demo của giai đoạn 2, các giá trị ghi bằng số cụ thể trong ví dụ API chỉ **minh họa định dạng**; chỉ các con số trích từ bài báo (ghi rõ) là mốc để so sánh.

2.1. Tái lập phương pháp của Golub (weighted voting)

Phụ trách: TV1 (trưởng nhóm) **Thời gian:** Tuần 5–6 (hoàn thiện tuần 7)

Hạng mục	Mô tả
Mục tiêu	Kiểm chứng kết quả gốc của bài báo và tạo baseline để so sánh với các phương pháp hiện đại.
Công việc cụ thể	- Cài đặt weighted voting: mỗi gen bỏ phiếu $v = a(x - b)$, với a là SNR và b là trung điểm hai trung bình lớp; tính prediction strength (PS) và ngưỡng 0.3. - Đóng gói thành classifier tương thích scikit-learn (fit, predict, predict_proba) trong src/golub_wv.py. - Tái lập LOOCV trên 38 mẫu (chọn lại 50 gen ở mỗi lần) và dự đoán 34 mẫu test. - Thử số gen từ 10 đến 200; vẽ strip plot PS theo mẫu. - Ghi mọi điểm khác biệt so với bài báo và giải thích nguyên nhân.
Phụ thuộc / phối hợp	Dùng gene_ranking của TV4 và evaluate.py bản 1 của TV2; giao classifier cho TV3.
Sản phẩm bàn giao	src/golub_wv.py; notebook 05_golub_reproduction; bảng so sánh với bài báo; hình PS; phần tái lập trong mục 3.4.

Demo kết quả để nghiệm thu

```
>>> from src.golub_wv import GolubWeightedVoting
>>> clf = GolubWeightedVoting(n_genes=50, ps_threshold=0.3).fit(X_train, y_train)
>>> clf.predict(X_test.iloc[:3])          # giá trị minh họa
array(['ALL', 'AML', 'uncertain'], dtype=object)
>>> clf.prediction_strength(X_test.iloc[:3])
array([0.81, 0.64, 0.12])
```

Bảng so sánh với bài báo:

Sơ đồ	Bài báo Golub (1999)	Nhóm tái lập
LOOCV (38)	36 dự đoán chắc chắn, 36/36 đúng	... chắc chắn, ... đúng
Test (34)	29 dự đoán chắc chắn, 29/29 đúng	... chắc chắn, ... đúng

Khác biệt và nguyên nhân: ...

2.2. Chọn biến và độ ổn định

Phụ trách: TV4 **Thời gian:** Tuần 5–7

Hạng mục	Mô tả
Mục tiêu	So sánh các phương pháp chọn gen và tìm ra một tập gen nhỏ, ổn định.
Công việc cụ thể	- Cài đặt các bộ chọn biến dưới dạng transformer: SNR, t-test, LASSO, Elastic Net, Random Forest importance, SVM-RFE; một hàm make_selector(tên, k) dùng chung. - Chạy chọn biến trên 100 mẫu bootstrap; tính tần suất chọn từng gen và Jaccard trung bình; vẽ UpSet plot (F7). - Chốt danh sách gen cuối (khoảng 10–50 gen) kèm độ ổn định.
Phụ thuộc / phối hợp	Kế thừa gene_ranking từ 1.6; giao bộ chọn cho TV3 và TV2; giao danh sách gen cuối cho TV5.
Sản phẩm bàn giao	src/features.py; notebook 06_selection; results/gene_lists/selected_*.csv, final_genes.csv; hình F7; mục 3.3.

Demo kết quả để nghiệm thu

```
>>> from src.features import make_selector
>>> sel = make_selector('lasso', k=20).fit(X_tr, y_tr)
```

```
>>> sel.transform(X_tr).shape
(n_train, 20)

Bảng độ ổn định (điền kết quả thực tế):
method k jaccard_mean số gen có tần suất chọn ≥ 80%
snr 20 ... ..
lasso 20 ... ..
svm_rfe 20 ... ..

>>> pd.read_csv('results/gene_lists/final_genes.csv').columns.tolist()
['probe_id', 'description', 'selection_freq', 'n_methods_agree', 'in_golub_50']
```

2.3. Các mô hình phân loại và thí nghiệm giai thừa

Phụ trách: TV3 **Thời gian:** Tuần 5–7

Hạng mục	Mô tả
Mục tiêu	Xây dựng và so sánh các bộ phân loại để trả lời H2 và H4.
Công việc cụ thể	- Cài đặt DLDA, k-NN, logistic L1/L2, SVM tuyến tính, Random Forest trong cùng Pipeline với tiền xử lý và chọn biến; thêm weighted voting của TV1. - Tuần 5 dùng SelectKBest tạm thời, thay bằng bộ chọn của TV4 khi có. - Chạy thí nghiệm theo experiment_config.yaml: 6 phương pháp × 6 mức k × 7 classifier = 252 cấu hình; ghi runs.parquet. - Chọn mô hình cuối, huấn luyện trên 38 mẫu, dự đoán 34 mẫu test; xuất dự đoán theo mẫu cho TV5.
Phụ thuộc / phối hợp	Dùng transformer của TV3 (1.2), TV4 (2.2), classifier của TV1 (2.1), evaluate.py của TV2 (2.4).
Sản phẩm bàn giao	src/models.py; notebook 07_models; results/metrics/runs.parquet; predictions_test.csv; số liệu cho F8, F9; mục 3.4.

Demo kết quả để nghiệm thu

```
>>> runs = pd.read_parquet('results/metrics/runs.parquet')
>>> runs.groupby(['selection', 'k', 'classifier']).ngroups
252
>>> runs.columns.tolist()
['run_id', 'selection', 'k', 'classifier', 'cv_scheme', 'seed',
'bal_acc', 'auc', 'sensitivity', 'specificity', 'n_uncertain']

Top 5 cấu hình theo balanced accuracy trung bình (nested CV): ...
Mô hình cuối được chọn: ... (lý do: ...)
>>> pd.read_csv('results/metrics/predictions_test.csv').columns.tolist()
['sample_id', 'true_class', 'pred_class', 'prob_aml', 'model']
```

2.4. Khung đánh giá và thí nghiệm selection bias

Phụ trách: TV2 **Thời gian:** Tuần 5–7

Hạng mục	Mô tả
Mục tiêu	Đảm bảo mọi con số hiệu năng là trung thực và chứng minh bằng thực nghiệm tác hại của đánh giá sai (H3).
Công việc cụ thể	- Viết evaluate.py với hàm evaluate(pipeline, X, y, scheme) cho 4 sơ đồ: original_split (38/34), loocv, nested_cv, wrong_cv. - Tính balanced accuracy, AUC, sensitivity, specificity, khoảng tin cậy Wilson; kiểm định McNemar cho cặp mô hình trên tập test. - Thí nghiệm selection bias: 100 lần hoán vị nhân × {CV đúng, CV sai}; vẽ F10. - Vẽ ROC và ma trận nhầm lẫn cho mô hình cuối (F11). - Review code của TV1, TV3, TV4 để phát hiện rò rỉ dữ liệu.
Phụ thuộc / phối hợp	Dùng experiment_config.yaml của TV1; hàm đánh giá được TV1 và TV3 dùng từ tuần 5.

Hạng mục	Mô tả
Sản phẩm bàn giao	src/evaluate.py; notebook 08_evaluation; hình F10, F11; bảng tổng hợp chỉ số; mục 3.5.

Demo kết quả để nghiệm thu

```
>>> from src.evaluate import evaluate
>>> res = evaluate(pipe, X, y, scheme='nested_cv', seeds=range(10))
>>> res.keys()
dict_keys(['bal_acc_mean', 'bal_acc_sd', 'auc_mean', 'per_fold', 'config'])
```

Bảng selection bias (điền kết quả thực tế; cột cuối là kỳ vọng):

Nhân	Cách CV	Balanced accuracy (TB $\pm$ SD)	Kỳ vọng
Thật	Đúng	... $\pm$...	cao
Thật	Sai	... $\pm$...	cao, có thể cao hơn CV đúng
Hoán vị	Đúng	... $\pm$...	xấp xỉ 0.50
Hoán vị	Sai	... $\pm$...	cao bất thường (bằng chứng H3)

2.5. Diễn giải sinh học và phân tích lỗi

Phụ trách: TV5 **Thời gian:** Tuần 6–7 (chuẩn bị từ tuần 5)

Hạng mục	Mô tả
Mục tiêu	Giải thích ý nghĩa sinh học của các gen được chọn và hiểu vì sao mô hình sai hoặc không chắc chắn.
Công việc cụ thể	- Tuần 5–6: tra cứu chức năng top 50 gen trong gene_ranking.csv trên NCBI Gene hoặc GeneCards. - Khi có final_genes.csv: chú giải từng gen, đánh dấu gen đặc trưng dòng tủy/dòng lympho, đối chiếu với các gen bài báo nêu (CD33, CD11c, MB-1, zyxin, leptin receptor, Cyclin D3, MCM3...). (Tùy chọn) Enrichment GO/KEGG qua thư viện gseapy (Enrichr). - Phân tích mẫu test dự đoán sai hoặc không chắc chắn theo tissue, source, subtype (F12); liên hệ với confounding ở 1.3. - Viết mục 3.6, phần thảo luận và hạn chế.
Phụ thuộc / phối hợp	Nhận final_genes.csv từ TV4 (CN 15/11), predictions_test.csv từ TV3 và PS từ TV1 (CN 15/11).
Sản phẩm bàn giao	Notebook 09_interpretation; results/gene_lists/gene_annotations.csv; hình F12; mục 3.6, thảo luận, hạn chế.

Demo kết quả để nghiệm thu

gene_annotations.csv (một dòng minh họa):

probe_id	symbol	chức năng tóm tắt	dòng tế bào	trong 50 gen Golub	nguồn
X95735_at	ZYX	protein liên quan bám dính tế bào	...	có	NCBI Gene

Bảng phân tích lỗi (điền kết quả thực tế):

sample_id	true	pred	prob/PS	tissue	source	subtype	giải thích khả dĩ
...	AML	ALL	...	PB	St-Jude	AML	...
...	ALL	uncertain	...	...	...	T-ALL	...

Kết luận: tỷ lệ lỗi/không chắc chắn ở mẫu PB và St-Jude so với phần còn lại: ...

2.6. Trực quan hóa kết quả và dashboard

Phụ trách: TV6 **Thời gian:** Tuần 5–8

Hạng mục	Mô tả
Mục tiêu	Biến các kết quả thành hình rõ ràng, nhất quán và đúng nguyên tắc trực quan hóa đã học.

Hạng mục	Mô tả
Công việc cụ thể	- Tuần 5: dựng khuôn cho F7–F12 bằng dữ liệu giả, theo định dạng file của các thành viên (runs.parquet, final_genes.csv...). - Tuần 6–7: thay dữ liệu thật, hoàn thiện F1–F12; review từng hình bằng checklist mục 4.1. - Xuất hai bản: báo cáo (độ phân giải cao) và slide (chữ to, đơn giản hóa); cùng TV1 thiết kế slide. (Tùy chọn) Dashboard Streamlit 3 tab: Gene explorer, Model lab, Patient view.
Phụ thuộc / phối hợp	Nhận số liệu từ mọi thành viên; làm việc sát với TV1 để mỗi hình phục vụ một thông điệp trong storyboard.
Sản phẩm bàn giao	results/figures/report/, results/figures/slides/; dashboard (tùy chọn); mục 4.1, 4.2, 4.4.

Demo kết quả thử nghiệm thu

Checklist nghiệm thu (mỗi hình một dòng, TV1 đánh dấu):

Hình Style guide Trục/đơn vị Thang màu đúng loại Mã hóa dư thừa Nhấn trực tiếp Duyệt

F01 ☒ ☒ ☒ ☒ ☒ ☐

...

F12 ☐ ☐ ☐ ☐ ☐ ☐

Quy ước tên file: F09_classifier_comparison_report.png

F09_classifier_comparison_slide.png

\$ make figures # vẽ lại toàn bộ F1–F12 từ results/ không lỗi

Tài liệu tham khảo chính

- Golub, T. R., et al. (1999). Molecular classification of cancer: Class discovery and class prediction by gene expression monitoring. *Science*, 286(5439), 531–537.
- Dudoit, S., Fridlyand, J., & Speed, T. P. (2002). Comparison of discrimination methods for the classification of tumors using gene expression data. *Journal of the American Statistical Association*, 97(457), 77–87.
- Ambrose, C., & McLachlan, G. J. (2002). Selection bias in gene extraction on the basis of microarray gene-expression data. *PNAS*, 99(10), 6562–6566.
- Guyon, I., Weston, J., Barnhill, S., & Vapnik, V. (2002). Gene selection for cancer classification using support vector machines. *Machine Learning*, 46, 389–422.
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *JRSS Series B*, 58(1), 267–288.
- Zou, H., & Hastie, T. (2005). Regularization and variable selection via the elastic net. *JRSS Series B*, 67(2), 301–320.
- Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate. *JRSS Series B*, 57(1), 289–300.
- Cleveland, W. S., & McGill, R. (1984). Graphical perception. *JASA*, 79(387), 531–554.
- Mackinlay, J. (1986). Automating the design of graphical presentations of relational information. *ACM Transactions on Graphics*, 5(2), 110–141.
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*. O'Reilly.
- Tài liệu bài giảng của bộ môn: HDFS; OLTP và OLAP; Data lake; Data integration and preprocessing; Exploratory data analysis; Introduction to data visualization; Visual model và visual encoding; Graphical perception; Visualization for table data; Principles of figure design.